

Development Plan

Software Engineering

Team 6, Blockchainerz

Robert Lam

Nathan Hum

Aidan Lao

Keira Laskoski

Table 1: Revision History

Date	Developer(s)	Change
September 27	Nathan	Changed project decomposition and scheduling, changed GitHub Projects usage, changed coding standards, changed Appendix – Team Charter
September 27	Aidan	Added sections 5 to 7 of the development plan regarding Team communication plan, roles, and workflow.
September 28	Robert	Added sections 1-4 and 9-10

This document outlines the processes, tools, and practices that the team will use to plan, develop, and manage the [DAIIM](#) project. It describes the team’s approach to communication, collaboration, project scheduling, workflow, and software development, as well as the technologies and coding standards that will guide implementation. The plan also identifies the project’s proof-of-concept strategy and describes how generative AI and other development tools will be incorporated into the development process.

1 Confidential Information?

This project does not contain any confidential or protected information.

2 IP to Protect

The team does not have any project-specific intellectual property that requires protection through an IP agreement. The DAIIM source code will be released as open-source software under the license described in the copyright license section below.

3 Copyright License

This project, including all source code and documentation, will be under the MIT license. The [license](#) can be found in the project repository.

4 Team Meeting Plan

Table 2: Team and Supervisor Meeting Schedule

Meeting	Location	Frequency	Time
Team meeting	McMaster campus	Weekly	Wednesday 1:30 pm
Supervisor meeting	Microsoft Teams	Biweekly	TBD

All meetings will be recorded in a Github issue outlining the date, attendance and agenda. Each meeting will have a chair who is responsible for creating the agenda and sharing it with others prior to the meeting. If possible the agenda should be shared 12hrs in advance to give team members a chance to review and propose different topics. For each meeting, the chair will ask for a volunteer from the team to record meeting minutes. If no one volunteers, the note-taking role will rotate among team members. Meeting minutes and decisions made during the meeting will be recorded as a comment on the meeting issue. Ad

hoc meetings may also be scheduled as necessary for urgent decisions, issue resolution, or preparation for a deliverable.

5 Team Communication Plan

The team will use a Microsoft Teams group chat for regular communication. The team also has a shared Word document for collaborative editing of documents and planning during meetings. Github issues will be used to track each meeting and any associated agendas/minutes.

Communication with the supervisor will be done through email and Microsoft Teams. Every email with the supervisor will be sent to all team members. The team will meet regularly with the supervisor, subject to the supervisor's availability.

Github issues will be the primary method of communication for development tasks. Each issue will represent a development task and will be assigned to a team member; the team member will be responsible for updating the issue with their progress. The issues will be associated with a milestone representing the deliverable, and will be placed appropriately in the Kanban project according to its status (e.g., To Do, In Progress, Done). Feedback is expected from at least one other team member for each issue, through either a comment on the Github issue or reviewed in a meeting or group chat.

6 Team Member Roles

- **Agenda creator:** The agenda creator is responsible for creating the agenda for a meeting, and posting it to the meeting issue. The agenda creator will use the template for team meetings located in the repo, containing the agenda and an attendance tracker.
- **Notetaker:** The notetaker is responsible for taking notes during a meeting, and posting the meeting minutes to the meeting issue. The meeting minutes will be posted as a comment under the meeting issue. Multiple people may contribute to the notetaker role during a meeting, especially if there are multiple topics / individual updates from each person.
- **Pull request reviewer:** The pull request reviewer role is a role that is assigned to a team member for each pull request. The pull request reviewer is responsible for reviewing the code changes in the pull request, providing feedback, and approving or requesting changes to the pull request. Every team member is expected to review pull requests, but the pull request reviewer is the primary reviewer for that pull request. They are in charge of making the final decision on approving the request.

- **Developer:** The developer role is a role assigned to a team member for each development task. The developer is responsible for implementing the task, and updating the associated issue with their progress. Every team member is expected to take on this role and contribute to development tasks as necessary, but the developer is the primary person responsible for completing a given task.

7 Workflow Plan

The following workflow plan can be used as a contributor's guide to the development process.

Issues and deliverables management: Each deliverable (and sub-deliverable if needed) will have a milestone associated with it. Each milestone will have a set of issues associated with it, representing the development tasks needed to complete the deliverable. These issues will be in the project Kanban board, and will be moved through the columns (To Do, In Progress, Done) as they are worked on.

Pull requests and code review: All code changes will be made in a new branch, and a pull request will be created once development is done for a particular task. See the branches and commit sections for naming convention. Typically, a branch will be created for each issue, but a branch may close several issues if they are related and can be completed together. Pull requests will be reviewed by at least one other team member, and the pull request reviewer role will be responsible for approving or requesting changes to the pull request.

7.1 Branches & Commits

Branch naming convention: Each branch will be named according to the following convention: `<branch-type>/<branch-name>`. Like mentioned, development will be done in a new branch, and a pull request will be created once development is done for a particular task. The naming convention for branches will be kebab-case and will be based on the branch content as well as the branch type. For example, a branch for the issue "Implement user login" would be named "feature/implement-user-login". The current types of branches are feature, docs, and bugfix, but more types may be added as needed.

Commit naming convention: Each commit will be named according to the following convention: `<commit-type>: <commit-message>`. The naming convention for commits will be lowercase and will be based on the commit content as well as the commit type. For example, a commit for the issue "Implement user login" would be named "feat: implement user login". The current types of commits are feat, fix, docs, style, refactor, and test, but more types may be added as needed.

7.2 Merging strategy

Developers will merge their changes into the main branch through pull requests. A pull request must be approved by at least one other team member before it can be merged into the main branch. Individual commits will be preserved in the pull request, and the pull request reviewer will be responsible for approving or requesting changes to the pull request.

8 Project Decomposition and Scheduling

The project is organized around key milestones and overarching stages (refer to Table 2), progressing from initial project planning and requirements analysis through design, implementation, verification, and final demonstration. Each stage contains defined deliverables and deadlines to provide the team with a clear progression throughout the project. The schedule will remain flexible where necessary, allowing the team to adjust tasks and responsibilities based on project needs, progress, and individual strengths.

Stage	Milestone	Deadline
Planning	Project selection	W2
Planning	Problem Statement + PoC + Development Plan + Team Charter	W4
Requirements	SRS + Hazard Analysis	W6
Verification	V&V Plan	W8
Design	Design Document	W10
Design	PoC Demo	W11
Implementation	Design Document Rev 0	W16
Implementation	Module Implementation	W16
Implementation	Rev 0 Demo	W18
Finalization	Final Demo + Documentation	W21
Finalization	V&V Report + CD	W22
Finalization	Final Demos	W23
Expo	Expo Materials	W25
Expo	Capstone EXPO	W26

Table 3: Project Milestones and Schedule

8.1 GitHub Project Usage

The project board will follow a Kanban-style workflow with columns for Backlog, Ready, In Progress, In Review, and Done. The project will be organized into milestones representing the overarching deliverables required throughout each stage of the project. For example, the initial stage contains milestones for the Project Statement and Development Plan, with individual tasks associated with each milestone. Each task will be represented as a card on the board and

assigned to the appropriate team member. Cards will move through the Kanban columns as work progresses, allowing the team to track the status of individual tasks and overall milestone completion.

9 Proof of Concept Demonstration Plan

9.1 Project Risks

One main risk is designing an aggregation algorithm that combines predictions from multiple inference providers into an accurate final result. The algorithm must handle disagreements between providers and reduce the influence of dishonest or low-quality predictions. This is difficult because, for a given task, the network may not have a source of *ground truth* against which to compare the predictions. There is also a cold-start problem where initially there may not be enough trusted providers in the network to correctly evaluate one another.

Another risk is designing the incentive system and token economics so that the network is sufficiently decentralized and is resistant to attacks. The system must be fair and accessible for participants so that they are encouraged to take part in the network. The system must also be economically sustainable and ensure that the distribution of rewards does not create incentives to exploit the system or reduce the quality of the aggregated results.

Collusion attacks pose a significant risk to the system. A collusion attack occurs when multiple participants, such as inference providers, coordinate their actions to manipulate the aggregation algorithm, reputations, and reward distribution. For example, providers could submit similar inaccurate predictions to make their results appear correct and receive rewards. This is difficult to detect when there is no ground truth and when the colluding providers make up a large portion of the network.

9.2 POC Demonstration Plan

The proof of concept plan is to create a simulation of the DAIIM network that is completely off chain. The simulation will execute a sequence of rounds in which an inference task for a regression problem is executed by the network. Various different types of actors will participate in the network:

1. Honest: attempt to maximize rewards by providing accurate predictions consistently (with some noise)
2. Learners: gradually improve their predictions over time
3. Malicious: intentionally provide bad predictions
4. Collusion: A group of providers working together to skew the network

The simulation will start with a simple ML task (eg. predict y , $y = mx + b$), and each round some input data will be randomly generated. The different actors will be given some algorithm (good or bad algorithm depending on the actor) to determine their predictions given the input. The actors will submit their predictions, and the aggregation algorithm will determine the aggregated

result. Actor's reputations will be updated and rewards will be distributed after each round.

Another layer of this simulation is to economically model each actor. We will assign providers a cost for running their model, transaction fees for submitting predictions on-chain, and economic rewards or penalties for participating. These values will be calibrated relative to the costs and payments that we expect a provider to realistically encounter when performing an inference task. This will ensure that the simulation reflects realistic participation decisions rather than relying on arbitrary values. We will also vary these values to measure how sensitive participation and network performance are to changes in costs, fees, rewards, and penalties.

We will test various combinations of actors to evaluate the limits of the system. These scenarios will vary the proportions of different actors, as well as the number of providers in the network. These results will allow us to identify conditions under which the system remains reliable and economically viable, as well as conditions under which malicious or colluding providers can significantly influence the network.

9.3 Success Criteria and Failure Response

This proof of concept will be successful and show that the stated risk can be overcome if: - The system provides accurate aggregated results that improve over time even with some limited number of malicious actors - It is more profitable to act honestly and attempt to provide good predictions than to be malicious - The network can be cold-started with the assumption that initially there are a majority of honest providers

If the proof of concept is not able to demonstrate that these risks can be addressed then the project's assumptions and scope may need to be changed. Before proceeding to full implementation, the team will review the findings with the supervisor and decide whether to revise the design, broaden the threat conditions, or reconsider the project's feasibility.

10 Expected Technology

This section describes the technologies that are expected to support the implementation of DAIIM. Note that these choices are preliminary and subject to change later in the project.

10.1 Core Implementation Stack

DAIIM will use Solana as the blockchain network for recording transactions and executing on-chain logic. During development, the team will use Solana Devnet, a separate network for testing programs and transactions without affecting Solana Mainnet. We will request free test SOL from a Devnet faucet to pay for transaction fees. Since Devnet tokens have no real-world value, the

team can develop and test DAIIM without using real money before deploying to Mainnet.

Solana programs, or smart contracts, will be written in Rust. The conventional off-chain backend will use Node.js with TypeScript and PostgreSQL for data storage. The user-facing interface will be developed with React.js and Vite. Python may also be used to write simple machine learning models for integration testing purposes.

10.2 Libraries and External Components

The expected external libraries and components are:

- *[Anchor]*: A development framework for building, deploying and testing Solana programs in Rust.
- *[Metamask]*: Used for accessing crypto wallets & signing transactions.
- *[Metaplex]*: Protocol and SDK for creating NFTs.
- *[@solana/web3.js]*: Javascript library for interacting with the Solana blockchain.
- *[@solana/spl-token]*: JavaScript library for interaction with the SPL token program on Solana.
- *[Vitest]*: Testing framework for the React frontend and other TypeScript code.
- *[Solana Devnet JSON-RPC]*: Public RPC access for testing blockchain transactions and programs.

10.3 Version Control and CI/CD

Git and Github will be used for version control and the project repository. GitHub Projects will organize issues and track their progress. LaTeX will be used for the project's technical documentation.

GitHub Actions will compile the LaTeX documentation automatically when relevant changes are pushed or proposed. Later, the CI pipeline will be extended to build and run tests for the solana program (using Anchor) and React frontend.

11 Use of AI and LLMs

[This section is a continuation of the previous section. The role of LLMs in your project as important enough to warrant their own section. How will your team be using LLMs for code and documentation development? What tools will you be using? How will you verify the output of your LLMs? —SS]

12 Coding Standard

The team will adhere to the coding standards outlined in Table 4.

Category	Details
React Frontend	React code will follow the official React documentation and established JavaScript/TypeScript conventions. Code will be formatted using Prettier and linted using ESLint .
Rust Backend	Rust code will follow the official Rust API Guidelines and community style conventions, formatted using rustfmt and linted with clippy .
Rust Documentation	Rust functions, structs, and modules will use rustdoc doc comments (<code>///</code> and <code>//!</code>) following the conventions in the official Rust documentation guide .
Docker	Dockerfiles and container images will follow Docker's official best practices for maintainability, efficiency, and security.
Git	Git will be used for version control, with feature branches used to isolate development work.
Code Review	Changes will be submitted through GitHub pull requests and reviewed by another team member before being merged into the main branch. See the GitHub pull request review guidelines .

Table 4: Coding Standards

13 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

[This section is about the use of AI to write this document, not your planned use of AI for your project. You discuss that topic in one of the sections in the body of this report. —SS]

13.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

13.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

13.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

Appendix — Reflection

[Not required for CAS 741 —SS]

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. Why is it important to create a development plan prior to starting the project?
2. In your opinion, what are the advantages and disadvantages of using CI/CD?
3. What disagreements did your group have in this deliverable, if any, and how did you resolve them?

Appendix — Team Charter

External Goals

For this Capstone, our team hopes to deliver a project that is both challenging and enjoyable to create. We aim to build something beyond simply fulfilling the requirements of a school project—something that we can be proud of and potentially carry forward into our professional careers. By focusing on producing a meaningful, well-developed, and high-quality project, we believe that a strong performance in the Capstone course will naturally follow.

Attendance

This section summarizes the ground rules and expectations regarding team member attendance.

Expectations

All team members are expected to attend weekly team meetings and meetings with our supervisor. Attendance at every Capstone course lecture is not required, provided that members remain up to date with the course material through either attending the lecture in person or reviewing the material independently.

Attendance at all required team and supervisor meetings will be tracked through GitHub Issues. All members are expected to maintain a 95% attendance rate throughout the project. When a meeting is held in person, team members should

make their best effort to attend in person. If a member is unable to attend in person, they should notify the team at least 5 hours in advance and attend virtually instead. If a member is unable to attend the meeting altogether, they must notify the team as soon as possible, preferably when they become aware of the conflict and at least 24 hours before the meeting when reasonably possible. Different expectations may apply depending on the circumstances.

Acceptable Excuse

Team members are expected to communicate with the rest of the team if they are unable to attend a meeting or meet a project deadline. Each member may miss up to three team meetings per term with prior notice. Acceptable reasons may include illness, family emergencies, religious observances, or previously arranged commitments that cannot reasonably be rescheduled.

If a member exceeds three absences in a term, the team will discuss the member's participation, expectations, and responsibilities. If the issue cannot be resolved

within the team, the team may involve the course professor or supervisor.

Team members are expected to meet all assigned deadlines. Exceptions will generally only be made in the event of an emergency, as outlined in the following section.

In Case of Emergency

If a team member is unable to meet a deadline or complete their assigned work due to an emergency, they must communicate this to the team as soon as they become aware that the work cannot be completed on time. The team will then

discuss appropriate accommodations and, if necessary, redistribute the affected work to ensure that project deadlines can still be met. If another team member must take on the incomplete work, the original team member will be expected to take on additional responsibilities during a subsequent phase, where appropriate, to help maintain a fair distribution of workload.

[borrows from [University of Portland Team Charter](#) —SS]

Accountability and Teamwork

Quality

[What are your team's expectations regarding the quality of team members' preparation for team meetings and the quality of the deliverables that members bring to the team? —SS]

Attitude

[What are your team's expectations regarding team members' ideas, interactions with the team, cooperation, attitudes, and anything else regarding team member contributions? Do you want to introduce a code of conduct? Do you want a conflict resolution plan? Can adopt existing codes of conduct. —SS]

Stay on Track

[What methods will be used to keep the team on track? How will your team ensure that members contribute as expected to the team and that the team performs as expected? How will your team reward members who do well and manage members whose performance is below expectations? What are the consequences for someone not contributing their fair share? —SS]

[You may wish to use the project management metrics collected for the TA and instructor for this. —SS]

[You can set target metrics for attendance, commits, etc. What are the consequences if someone doesn't hit their targets? Do they need to bring the coffee to the next team meeting? Does the team need to make an appointment

with their TA, or the instructor? Are there incentives for reaching targets early?
—SS]

Team Building

[How will you build team cohesion (fun time, group rituals, etc.)? —SS]

Decision Making

[How will you make decisions in your group? Consensus? Vote? How will you handle disagreements? —SS]